

Relatório 9 - Prática: Datasets Sintéticos com Objetos Reais (III)

Ana Clara Fortunato de Souza

Descrição da atividade

1. Introdução

A geração de dados sintéticos permite criar imagens rotuladas em ambientes virtuais controlados, reduzindo parte do esforço necessário para fotografar, organizar e anotar grandes conjuntos de dados reais. Nesta atividade, o conteúdo estudado reúne três etapas complementares: a utilização do Jupyter Notebook, o treinamento de um classificador de imagens com TensorFlow e a criação de uma cena tridimensional mais elaborada no Blender a partir de um objeto externo no formato FBX.

A prática proposta utiliza o modelo `tea_mug.fbx` para produzir imagens de uma caneca em quatro situações: vazia, quase vazia, pela metade e cheia. A posição da câmera, o enquadramento e pequenas propriedades visuais são variados automaticamente, formando conjuntos de treino, validação e teste. Em seguida, essas imagens são utilizadas para adaptar uma rede MobileNetV2 pré-treinada, estabelecendo um fluxo completo entre modelagem 3D, renderização automatizada e classificação por aprendizado de máquina.

2. Conteúdos estudados

2.1 Jupyter Notebook

O terceiro vídeo apresenta o Jupyter Notebook como um ambiente interativo no qual código, explicações em Markdown, gráficos e resultados podem ser mantidos no mesmo arquivo. A execução por células facilita a inspeção de cada etapa do experimento, desde o carregamento das imagens até a avaliação do modelo. Também se destaca a importância de iniciar o Jupyter no ambiente virtual correto, pois as bibliotecas instaladas precisam estar disponíveis para o kernel utilizado pelo notebook.

Para esta prática, o notebook foi empregado na organização do pipeline de classificação. As células foram separadas em carregamento do dataset, visualização de amostras, construção do modelo, treinamento, geração das curvas de desempenho e avaliação com matriz de confusão.

2.2 Classificação de imagens com TensorFlow

O quarto vídeo mostra a diferença entre classificação e detecção de objetos. Na classificação, cada imagem recebe um único rótulo; na detecção, o sistema também precisa localizar os objetos presentes. Como cada render da prática contém uma caneca centralizada e corresponde a apenas um nível de líquido, a classificação é adequada ao objetivo.

Foi adotada a transferência de aprendizado com MobileNetV2. Nessa abordagem, uma rede previamente treinada em um conjunto amplo de imagens atua como extratora de características, enquanto uma nova camada final aprende as quatro classes específicas da atividade. O congelamento inicial da base reduz o custo de treinamento e permite aproveitar padrões visuais já aprendidos, como bordas, texturas e formas. As imagens são

redimensionadas para 224 × 224 pixels e pré-processadas conforme os requisitos da MobileNetV2.

2.3 Criação do ambiente 3D no Blender

O quinto vídeo amplia a complexidade dos exemplos anteriores ao importar um modelo FBX e construir uma cena de estúdio ao seu redor. São abordados o fundo, a iluminação, os materiais, o enquadramento e o uso de constraints. A restrição Track To mantém a câmera apontada para um objeto-alvo mesmo quando sua posição é alterada pelo script, evitando que a caneca saia do centro da imagem.

O exemplo do chá também evidencia que materiais transparentes e líquidos exigem mais cuidado do que objetos sólidos. Propriedades como transmissão, rugosidade, absorção de volume, reflexão e refração afetam diretamente o realismo. Nesta primeira reprodução, foi utilizada uma representação simplificada do líquido, suficiente para diferenciar os níveis e tornar o dataset viável em computadores com menor capacidade de renderização.

3. Desenvolvimento da prática

Repositório com os arquivos.

3.1 Preparação da cena

O arquivo tea_mug.fbx foi definido como o objeto principal. O script remove os elementos da cena inicial, importa o modelo, calcula seus limites tridimensionais e ajusta automaticamente sua escala e posição. Em seguida, são adicionados um piso, um fundo vertical e duas luzes de área, que simulam uma iluminação de estúdio com sombras suaves. Uma câmera é criada com a constraint Track To direcionada a um Empty localizado no centro visual da caneca.

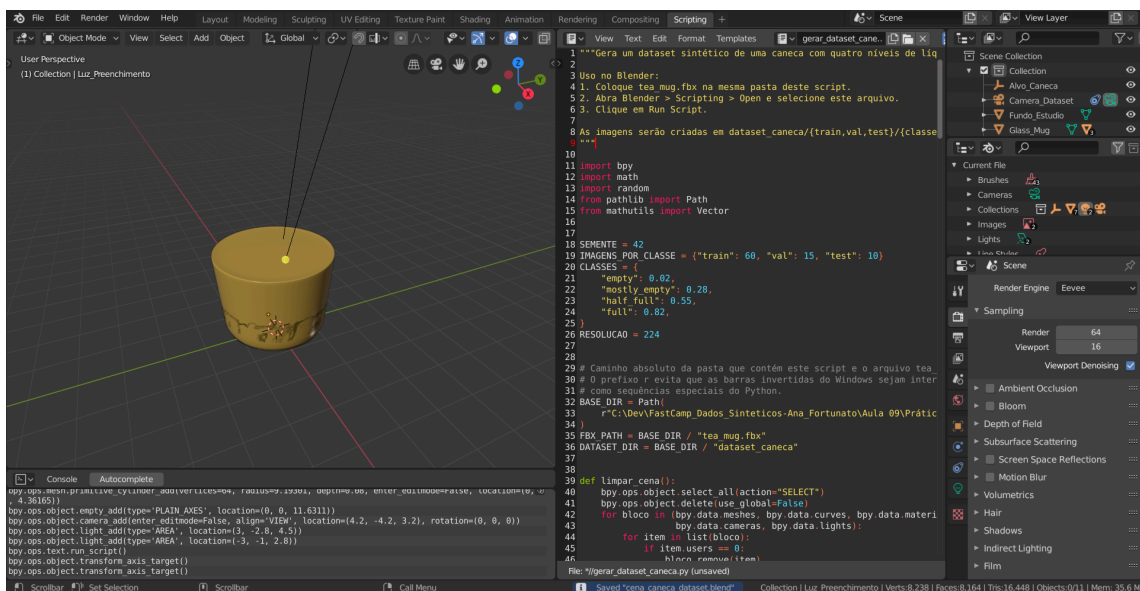


Figura 1 – Configuração da cena utilizada na geração do dataset sintético.

Fonte: elaboração própria (2026).

3.2 Classes e organização do dataset

Foram definidas quatro classes relacionadas ao nível de preenchimento da caneca. Em cada renderização, o script modifica o ângulo e a altura da câmera, a distância focal e pequenas

variações da cor do líquido. Essas mudanças aumentam a diversidade das amostras e diminuem a dependência do classificador em relação a um único enquadramento.

Classe	Nível	Treino	Validação	Teste
empty	Vazia	60	15	10
mostly_empty	Quase vazia	60	15	10
half_full	Metade	60	15	10
full	Cheia	60	15	10

Tabela 1 – Distribuição planejada das 380 imagens do dataset.

Fonte: elaboração própria (2026).

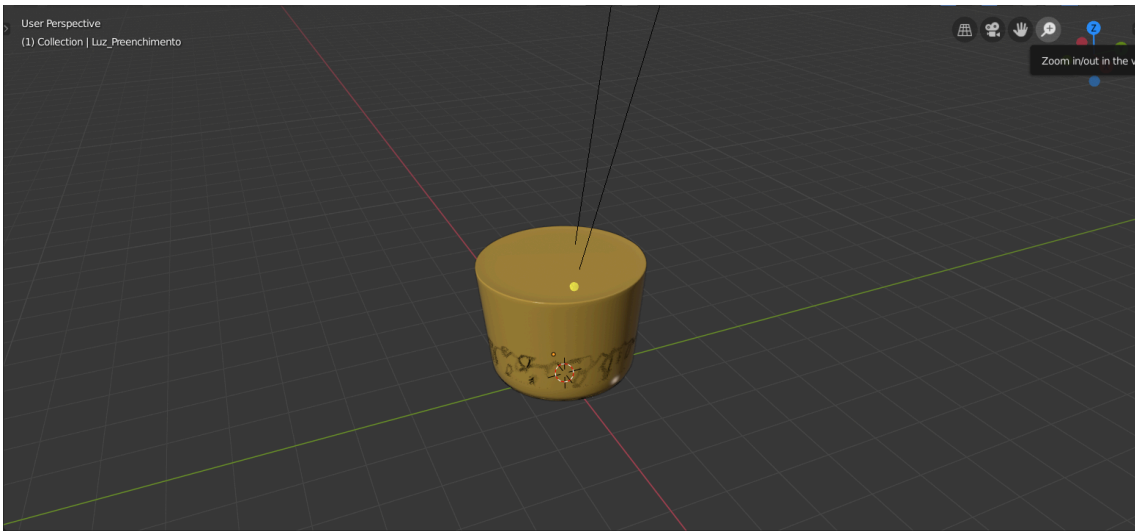


Figura 2 – Enquadramento utilizado para a renderização das imagens sintéticas.

Fonte: elaboração própria (2026).

3.3 Automação das renderizações

A automização foi realizada com a API Python do Blender, bpy. Laços de repetição percorrem as três divisões do dataset e as quatro classes. Antes de cada render, parâmetros da cena são sorteados dentro de intervalos controlados. O arquivo é salvo em uma pasta cujo nome corresponde ao rótulo da classe, permitindo que o TensorFlow carregue os dados diretamente a partir da estrutura de diretórios.

A semente aleatória foi fixada para tornar a execução reproduzível. O motor Eevee foi escolhido para reduzir o tempo de geração, mantendo resolução de 224 × 224 pixels. Essa decisão representa um equilíbrio entre qualidade visual e custo computacional, principalmente porque centenas de imagens precisam ser produzidas.

3.4 Treinamento e avaliação do classificador

No notebook, as imagens são carregadas com image_dataset_from_directory e organizadas em lotes. A base MobileNetV2 é inicializada com pesos do ImageNet, sem a camada classificadora original, e permanece congelada na primeira etapa. Após o agrupamento global

das características, uma camada densa com ativação softmax fornece as probabilidades das quatro classes. O modelo é compilado com o otimizador Adam e a função de perda sparse categorical crossentropy.

O treinamento utiliza parada antecipada para recuperar os melhores pesos quando a perda de validação deixa de melhorar. A avaliação final considera a acurácia no conjunto de teste e a matriz de confusão, que permite identificar quais níveis de líquido foram confundidos pelo modelo.

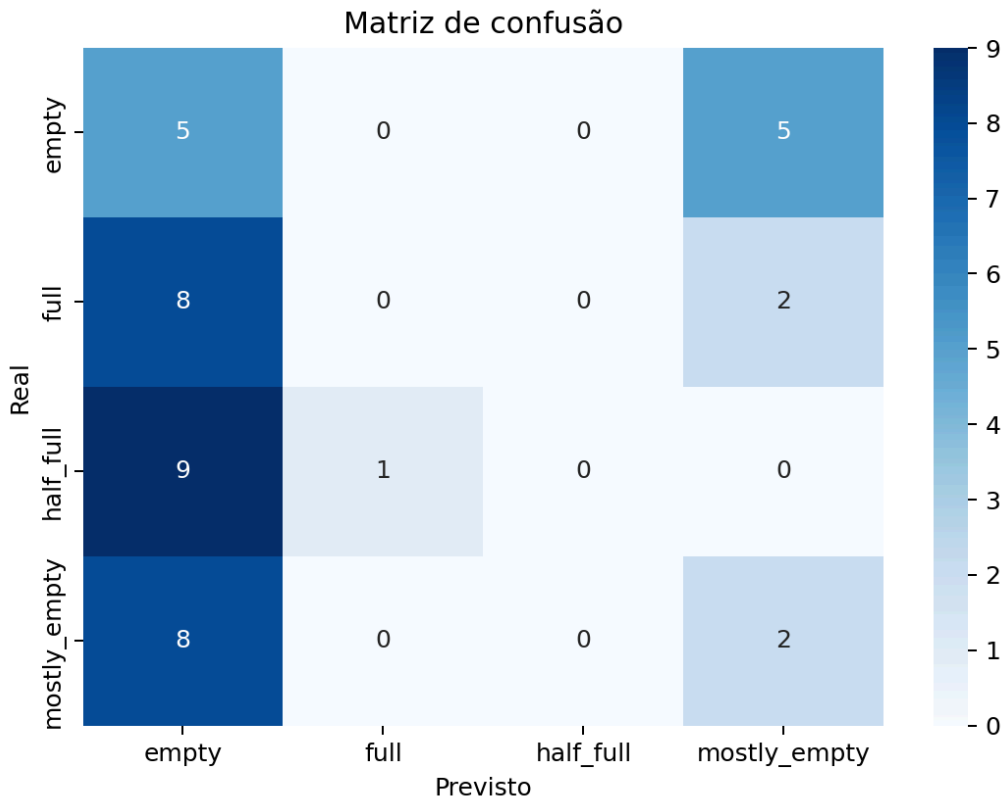


Figura 3 – Matriz de confusão do classificador no conjunto de teste.

Fonte: elaboração própria (2026).

4. Resultados

A análise dos resultados deve considerar não apenas a acurácia geral, mas também o comportamento por classe. Níveis próximos, como quase vazia e metade, tendem a apresentar maior semelhança visual e podem ser confundidos com mais frequência. Caso a matriz de confusão revele esse padrão, novas imagens com ângulos superiores, maior contraste do líquido ou diferenças mais evidentes de altura podem melhorar a separação entre as classes.

Outro ponto importante é a diferença entre imagens sintéticas e fotografias reais, conhecida como reality gap. Embora a randomização de câmera e cor aumente a variedade do dataset, fatores presentes no mundo real — reflexos imprevisíveis, ruído do sensor, fundos complexos e diferentes condições de iluminação — ainda podem reduzir o desempenho fora do ambiente simulado. Um teste futuro com fotografias reais da caneca seria necessário para avaliar a capacidade de generalização do modelo.

Dificuldades

A principal dificuldade ocorreu na configuração da cena e no enquadramento da câmera após a importação do arquivo FBX. Como a escala e a origem de modelos externos podem variar, foi necessário calcular os limites do objeto antes de posicionar os demais elementos. Outra dificuldade esteve relacionada ao tempo de renderização e à representação do líquido, pois materiais com transparência e refração exigem mais processamento. A utilização do Eevee e de um material simplificado permitiu reduzir esse custo sem comprometer o objetivo de classificação.

Conclusões

A atividade integrou as principais etapas de um projeto com dados sintéticos: preparação de um ambiente tridimensional, importação de um objeto externo, automatização da renderização, organização das imagens e treinamento de um classificador. O Jupyter Notebook facilitou a execução e a análise do experimento, enquanto o TensorFlow permitiu aplicar transferência de aprendizado sem treinar uma rede convolucional completamente do zero.

A prática com a caneca mostrou que a qualidade do dataset não depende apenas da quantidade de imagens, mas também da diversidade e da coerência das variações simuladas. Alterações de câmera, iluminação e aparência precisam representar situações possíveis, pois transformações artificiais ou pouco realistas podem prejudicar a generalização. Assim, Blender e Python formam uma combinação útil para gerar dados de maneira escalável, mas os resultados devem ser validados com imagens reais antes da aplicação do modelo em um cenário prático.

Referências

KELLY, Adam. **3D Rendered Datasets in Blender for Beginners, Part 3 [FREE COURSE]**.

YouTube. Disponível em: <https://www.youtube.com/watch?v=JrSn-90AHf8>. Acesso em: 5 ago. 2026.

KELLY, Adam. **3D Rendered Datasets in Blender for Beginners, Part 4 [FREE COURSE]**.

YouTube. Disponível em: <https://www.youtube.com/watch?v=6bK40srQpo4>. Acesso em: 5 ago. 2026.

KELLY, Adam. **3D Rendered Datasets in Blender for Beginners, Part 5 [FREE COURSE]**.

YouTube. Disponível em: <https://www.youtube.com/watch?v=a2YilEIFCvc>. Acesso em: 5 ago. 2026.

TENSORFLOW. **Transfer learning and fine-tuning**. Disponível em:

https://www.tensorflow.org/tutorials/images/transfer_learning. Acesso em: 5 ago. 2026.

BLENDER FOUNDATION. **Blender Python API Documentation**. Disponível em:

<https://docs.blender.org/api/current/>. Acesso em: 5 ago. 2026.